

LAB 2

TWO-TIER ARCHIECTURE

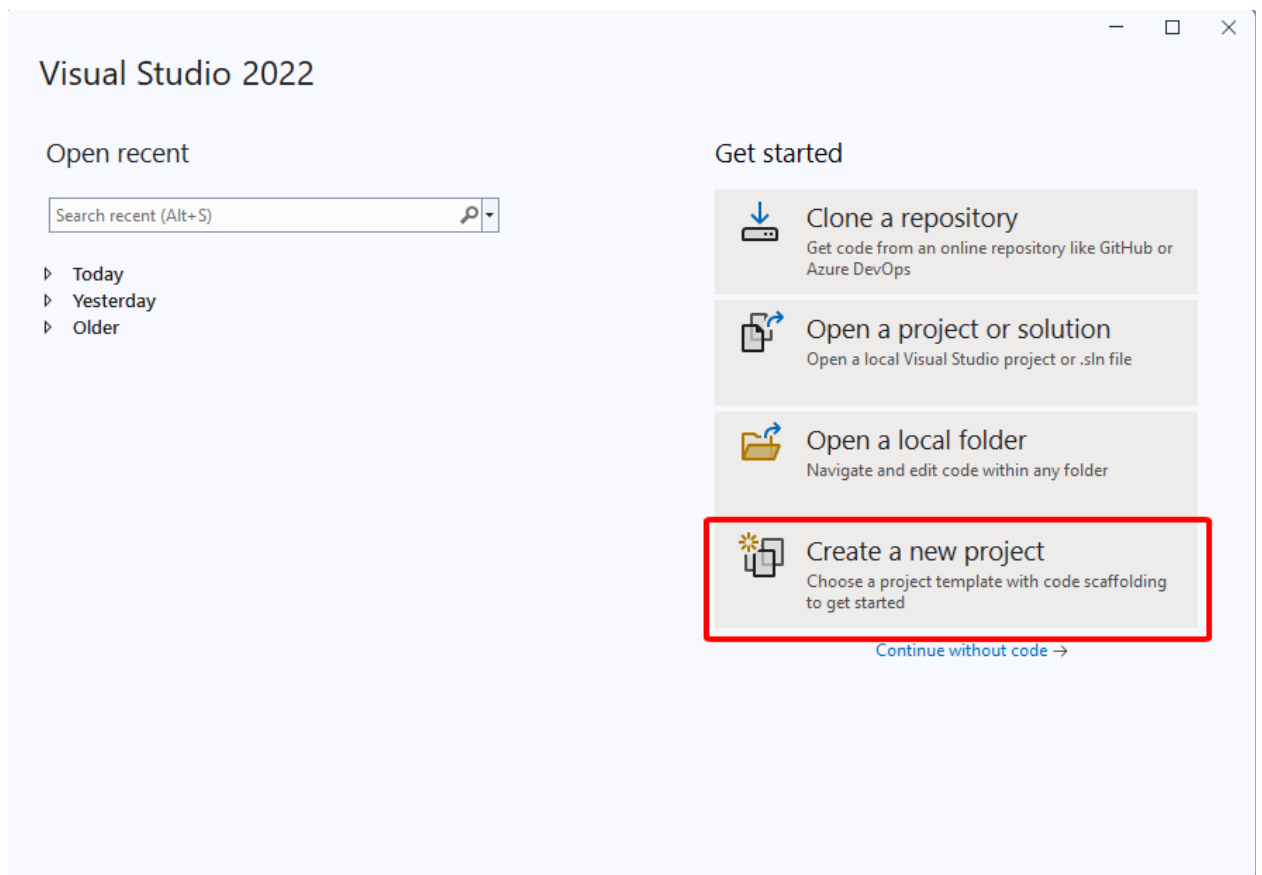
MỤC TIÊU

1. Xây dựng ứng dụng console dữ liệu được lưu trữ trên SQL Server
2. Sử dụng Dapper, SQL Connection truy vấn đến DB

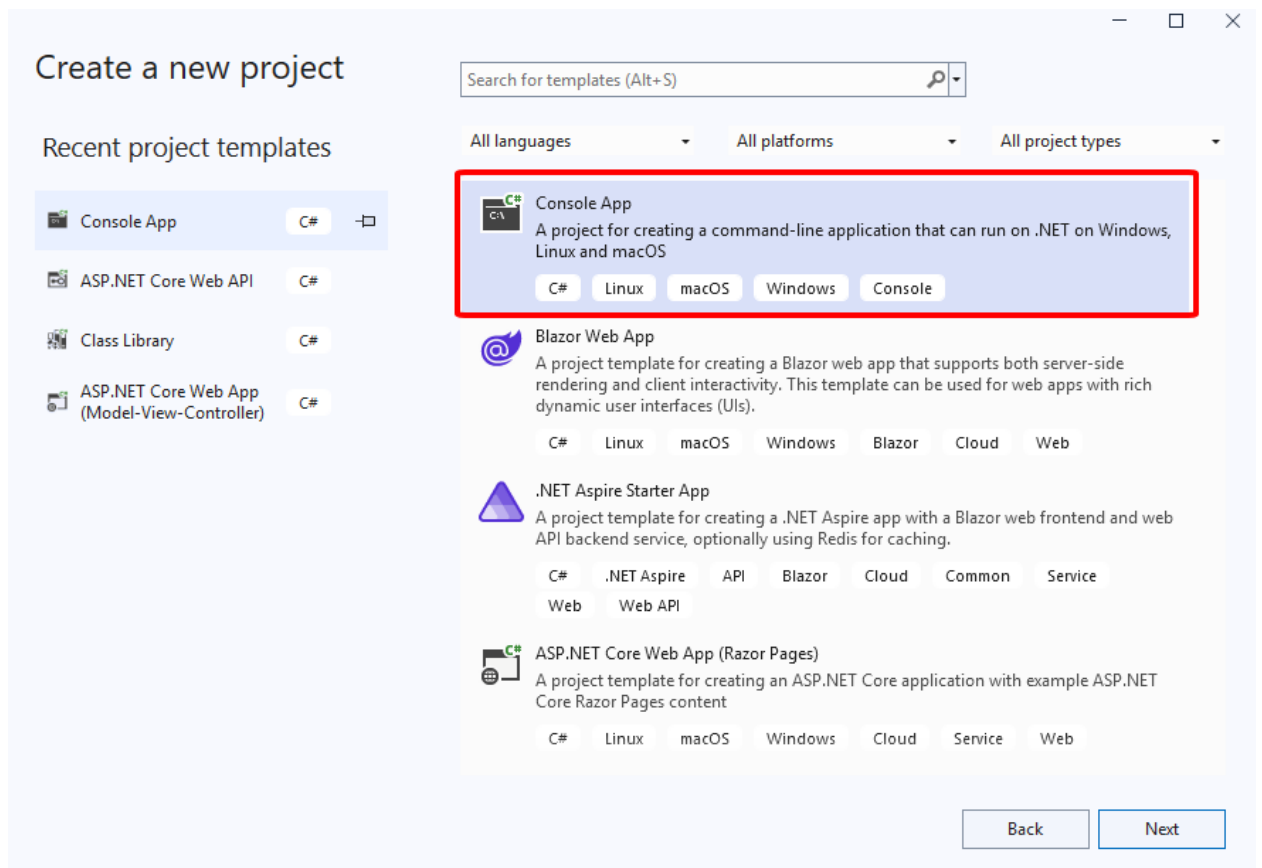
BÀI TẬP THỰC HÀNH: Xây dựng ứng dụng Todo trên Console (Tách UI – Logic – Data). Lưu trữ dữ liệu trên SQL Server

Bước 1. Tạo Project

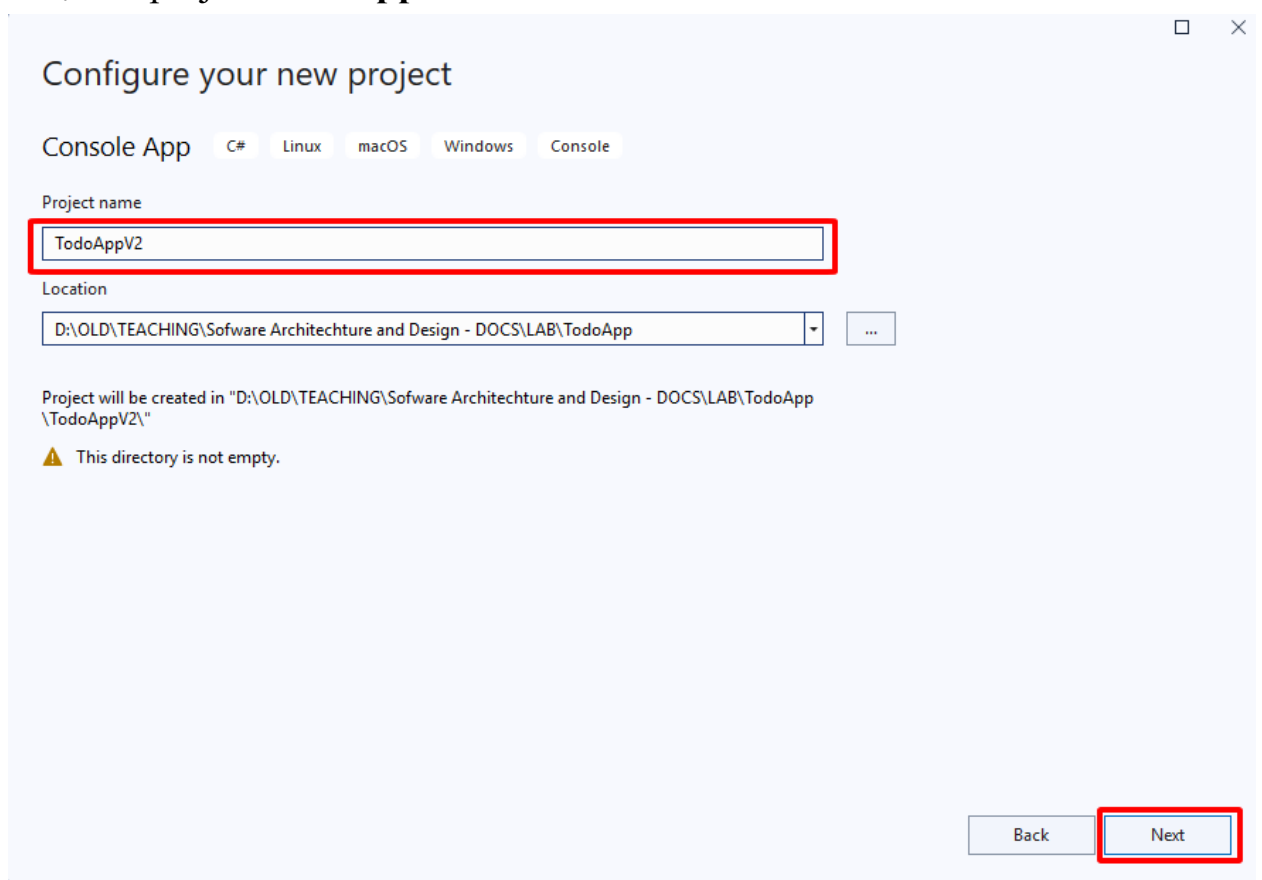
- **Mở Visual Studio 2022 => Chọn Create a new project**



- Chọn Template Console Application



- Đặt tên project **ToDoApp**



Bước 2. Tạo class Todo và code như sau

`using System;`

```

using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace TodoApp
{
    public class Todo
    {
        public int Id { get; set; }
        public string Title { get; set; }
        public bool IsCompleted { get; set; }

        public override string ToString()
        {
            return $"[{(IsCompleted ? "x" : " ")}] {Id}: {Title}";
        }
    }
}

```

Bước 3. Tạo class TodoRepository và code như sau

```

using Dapper;
using Microsoft.Data.SqlClient;
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace TodoAppV2
{
    public class TodoRepository
    {
        private readonly string _connectionString;
        public TodoRepository(string connectionString)
        {
            _connectionString = connectionString;
        }
        public SqlConnection ConnectDB()
        {
            SqlConnection database = new SqlConnection(_connectionString);
            return database;
        }
        public async Task<List<Todo>> GetAllAsync()
        {
            using (var database = ConnectDB()){
                var result = await database.QueryAsync<Todo>("SELECT *
FROM Todos");
                return result.ToList();
            }
        }
        public async Task<Todo> GetAsync(int id)
        {
            using (var database = ConnectDB())
            {
                var result = await database.QueryFirstAsync<Todo>("SELECT
* FROM Todos WHERE Id = @id", new {id});
                return result;
            }
        }
        public async Task AddAsync(string title)
        {
            using (var database = ConnectDB())

```

```

        {
            string query = @"INSERT INTO Todos (Title) VALUES
(@title)";
            var id = await database.ExecuteScalarAsync<int>(query, new
{ Title = title });
        }

        public async Task UpdateAsync(Todo todo)
        {
            using (var database = ConnectDB())
            {
                string query = @"UPDATE Todos
SET Title = @Title,
    IsCompleted = @IsCompleted
WHERE Id = @Id";

                var rowsAffected = await database.ExecuteAsync(query,
todo);
            }
        }

        public async Task DeleteAsync(int id)
        {
            using (var database = ConnectDB())
            {
                string query = @"DELETE FROM Todos WHERE Id = @id";
                await database.ExecuteAsync(query, new { Id = id });
            }
        }
    }
}

```

Bước 4. Tạo interface IToDoService và code như sau

```

using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace TodoAppV2
{
    public interface IToDoService
    {
        public Task<List<Todo>> GetAllAsync();
        public Task<Todo> GetAsync(int id);
        public Task addAsync(string title);
        public Task DeleteAsync(int id);
        public Task UpdateAsync(int id, string title);
        public Task ToggleTodoAsync(int id);
    }
}

```

Bước 5. Tạo class TodoService và code như sau

```

using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace TodoAppV2
{
    public class TodoService : IToDoService
    {

```

```

        private readonly IRepository<Todo> _repository;

        public TodoService(IRepository<Todo> repository)
        {
            _repository = repository;
        }

        public async Task<List<Todo>> GetAllAsync() => await
            _repository.GetAllAsync();

        public async Task addAsync(string title) => await
            _repository.AddAsync(title);

        public async Task DeleteAsync(int id) => await
            _repository.DeleteAsync(id);

        public async Task<Todo> GetAsync(int id) => await
            _repository.GetAsync(id);

        public async Task ToggleTodoAsync(int id)
        {
            Todo todo = await _repository.GetAsync(id);
            todo.IsCompleted = !todo.IsCompleted;
            await _repository.UpdateAsync(todo);
        }

        public async Task UpdateAsync(int id, string title)
        {
            Todo todo = await _repository.GetAsync(id);
            todo.Title = title;
            await _repository.UpdateAsync(todo);
        }
    }
}

```

Bước 6. Tạo class TodoUI và code như sau:

```

using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace TodoAppV2
{
    public class TodoUI
    {
        private readonly TodoService _todoService;
        public TodoUI(TodoService todoService)
        {
            _todoService = todoService;
        }

        public async Task Run()
        {
            while (true)
            {
                Console.Clear();
                await ShowTodos();
                ShowMenu();

                string choice = Console.ReadLine();
                switch (choice)
                {
                    case "1":

```

```

        AddTodo();
        break;
    case "2":
        DeleteTodo();
        break;
    case "3":
        ToggleTodo();
        break;
    case "4":
        EditTodo();
        break;
    case "0":
        return;
    default:
        Console.WriteLine("Lựa chọn không hợp lệ!");
        break;
    }

    Console.WriteLine("Nhấn Enter để tiếp tục...");
    Console.ReadLine();
}

private async Task ShowTodos()
{
    var todos = await _todoService.GetAllAsync();
    Console.WriteLine("=== DANH SÁCH CÔNG VIỆC ===");
    foreach (var todo in todos)
    {
        Console.WriteLine(todo);
    }

    if (todos.Count == 0)
        Console.WriteLine("Chưa có công việc nào.");
}

private void ShowMenu()
{
    Console.WriteLine("\nChức năng:");
    Console.WriteLine("1. Thêm Todo");
    Console.WriteLine("2. Xoá Todo");
    Console.WriteLine("3. Đánh dấu hoàn thành");
    Console.WriteLine("4. Sửa nội dung");
    Console.WriteLine("0. Thoát");
    Console.Write("Chọn: ");
}

private async Task AddTodo()
{
    Console.Write("Nhập nội dung công việc: ");
    string title = Console.ReadLine();
    if (!string.IsNullOrEmpty(title))
        await _todoService.addAsync(title);
}

private async Task DeleteTodo()
{
    Console.Write("Nhập ID công việc cần xoá: ");
    if (int.TryParse(Console.ReadLine(), out int id))
        await _todoService.DeleteAsync(id);
}

private async Task ToggleTodo()
{
    Console.Write("Nhập ID cần đánh dấu hoàn thành: ");
}

```

```

        if (int.TryParse(Console.ReadLine(), out int id))
            await _todoService.ToggleTodoAsync(id);
    }

    private async Task EditTodo()
    {
        Console.Write("Nhập ID cần sửa: ");
        if (int.TryParse(Console.ReadLine(), out int id))
        {
            Console.Write("Nhập nội dung mới: ");
            var newTitle = Console.ReadLine();
            if (!string.IsNullOrEmpty(newTitle))
                await _todoService.UpdateAsync(id, newTitle);
        }
    }
}

```

Bước 7. Mở file Program.cs và code như sau:

```

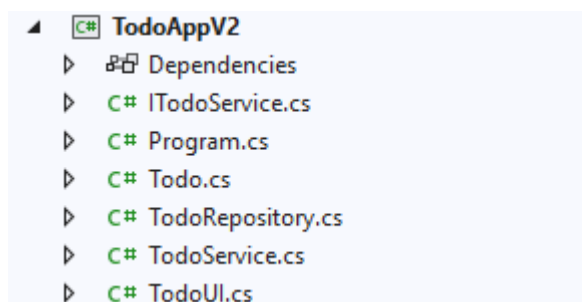
using System.Text;
using TodoAppV2;

public class Program
{
    private static async Task Main(string[] args)
    {
        Console.OutputEncoding = Encoding.UTF8;
        Console.InputEncoding = Encoding.UTF8;
        var repository = new TodoRepository("Server=DESKTOP-
OEF8EG\\SQLEXPRESS;Database=TodoDB;Integrated
Security=true;TrustServerCertificate=true");
        var service = new TodoService(repository);
        var ui = new TodoUI(service);
        await ui.Run();
    }
}

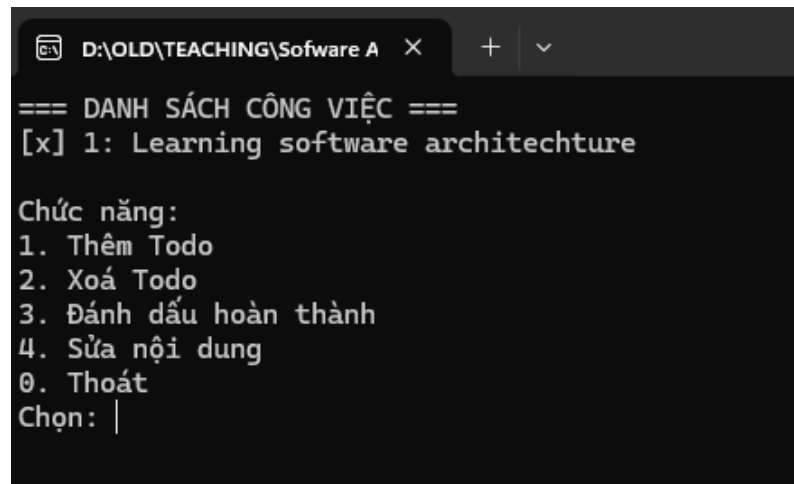
```

KẾT QUẢ

- Project



- Console



```
D:\OLD\TEACHING\Software A X + v

=== DANH SÁCH CÔNG VIỆC ===
[x] 1: Learning software architechture

Chức năng:
1. Thêm Todo
2. Xoá Todo
3. Đánh dấu hoàn thành
4. Sửa nội dung
0. Thoát
Chọn: |
```

BÀI TẬP

Xây dựng ứng dụng quản lý sinh viên trên Console theo yêu cầu:

- Tạo Class Student: Id, Name, Email, Address, Age, Grade
- Tách Code thành 3 phần riêng biệt UI – Logic – Data
- Dữ liệu được lưu trữ trên MongoDB
- Chức năng: Hiển thị danh sách sinh viên, thêm, sửa, xoá. Tìm kiếm sinh viên theo Id, Name, Address, Grade